DocGen — Complete Code Explanation

> For group members who didn't build this 😭

What is this project?

Topic: How automated tools help in recovering missing documentation from legacy systems

What we built: A full-stack desktop application that:

- Takes any Python source file as input
- Automatically analyzes its structure using Python's AST module
- Extracts all functions, classes, parameters, and docstrings
- Displays human-readable documentation in a clean JavaFX UI
- Allows export to Markdown format

Tech Stack:

- Frontend: JavaFX (Java desktop UI)
- Backend: Python FastAPI (REST API server)
- Code Analysis: Python `ast` module (built-in)
- JSON Parsing: Gson (Java library)
- HTTP Communication: Java's built-in `HttpClient`

Architecture Overview

```
User selects .py file
        ↓
JavaFX UI (MainController)
        ↓
ApiService.java builds multipart HTTP POST request
        ↓
FastAPI Backend (main.py) receives the file
        ↓
ast.parse() converts code into a tree structure
        ↓
ast.walk() extracts functions, classes, parameters
```

```
        ↓
Returns JSON response
        ↓
MainController formats JSON into readable documentation
        ↓
Displayed in TextArea + exportable as .md file
```

FILE 1: `main.py` (Python Backend)

Imports

```python
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
import ast
```

- `FastAPI` — the class that creates your server object
- `UploadFile` — the object representing the uploaded file you can read from
- `File` — tells FastAPI to expect a file in the request
- `CORSMiddleware` — allows the JavaFX app (running on a different port) to talk to the backend without being blocked by security policies
- `ast` — stands for Abstract Syntax Tree. Python's built-in module for reading and analyzing Python source code WITHOUT executing it

---

Server Creation + CORS

```python
app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- `app = FastAPI()` — creates the server. this is the object uvicorn looks for when you run `uvicorn main:app`
- `add_middleware(CORSMiddleware)` — attaches a security guard to every request
- `allow_origins=["*"]` — allow requests from ANY source
- `allow_methods=["*"]` — allow any HTTP method (GET, POST, etc.)
- `allow_headers=["*"]` — allow any headers
- The `*` means "everyone is allowed" — fine for local development

---

GET vs POST (Important Concept)

GET	POST
Fetching/reading data	Sending data to server
Data goes in the URL	Data goes in request body
Example: opening a webpage	Example: uploading a file

```python
@app.get("/")          # Just checking if server is alive
def root():
    return {"message": "DocGen backend is running"}

@app.post("/analyze")  # Sending a file to be analyzed
async def analyze(file: UploadFile = File(...)):
```

- `async` — means this function can pause and wait (for file reading) without freezing the entire server
- `@app.post("/analyze")` — this decorator registers the function as a POST endpoint at the `/analyze` URL

`extract_docs()` — The Core Function

```python
```

```python
def extract_docs(source_code: str):
    tree = ast.parse(source_code)
    result = []
```

- `source_code: str` — receives the uploaded file as a plain text string
- `ast.parse(source_code)` — converts the text into a tree data structure. every function, class, variable becomes a NODE in the tree
- `result = []` — empty list we'll fill with extracted info

```python
for node in ast.walk(tree):
```

- `ast.walk(tree)` — visits every single node in the tree one by one. like going through every room in a building

```python
    if isinstance(node, ast.ClassDef):
        class_doc = ast.get_docstring(node) or "No description available."
        methods = []
```

- `isinstance(node, ast.ClassDef)` — "is this node a class definition?"
- `ast.get_docstring(node)` — tries to find a docstring (the string written right after a class/function definition). falls back to "No description available." if none exists

```python
        for item in node.body:
            if isinstance(item, ast.FunctionDef):
                params = [arg.arg for arg in item.args.args if arg.arg != "self"]
```

- `node.body` — everything inside the class (methods, variables, etc.)
- `[arg.arg for arg in item.args.args if arg.arg != "self"]` — list comprehension that extracts all parameter names, skipping `self` because it's not a real parameter

```python
```

```
            methods.append({
                "name": item.name,
                "params": params,
                "docstring": method_doc,
                "line": item.lineno
            })
```

- `item.name` — function name as a string
- `item.lineno` — line number where this function starts in the original file
- packages everything into a dictionary and adds to methods list

---

FILE 2: `ApiService.java`

Purpose

This class is the **messenger** between JavaFX and FastAPI. It takes a file, wraps it in the correct HTTP format, sends it to the backend, and returns the JSON response.

Key Constant

```java
private static final String BACKEND_URL = "http://127.0.0.1:8000/analyze";
```

- `private` — only this class can access it
- `static` — belongs to the class itself, not any instance
- `final` — value can never change (it's a constant)
- `127.0.0.1` = localhost (your own machine), `8000` = uvicorn's port

What is Multipart Form Data?

When you upload a file on any website, it doesn't send raw bytes. It wraps the file in a format called `multipart/form-data` that looks like this:

```
--boundary123
Content-Disposition: form-data; name="file"; filename="main.py"
Content-Type: text/plain

<actual file contents here>
--boundary123--
```

FastAPI expects this exact format. We build it manually in Java.

```java
String boundary = UUID.randomUUID().toString();
```

- Generates a random unique string used as the separator between parts. Must be unique so it doesn't accidentally appear inside the file content.

```java
byte[] fileBytes = Files.readAllBytes(file.toPath());
```

- Reads the entire `.py` file from disk as a byte array. Everything in HTTP is sent as bytes.

```java
byte[] body = new byte[header.length + fileBytes.length + footer.length];
System.arraycopy(header, 0, body, 0, header.length);
System.arraycopy(fileBytes, 0, body, header.length, fileBytes.length);
System.arraycopy(footer, 0, body, header.length + fileBytes.length, footer.length);
```

- Creates one big byte array combining: header + file bytes + footer
- `System.arraycopy()` copies each part into the big array like stacking three pieces of paper

---

Sending the Request

```java
```

```
HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(BACKEND_URL))
        .header("Content-Type", "multipart/form-data; boundary=" + boundary)
        .POST(HttpRequest.BodyPublishers.ofByteArray(body))
        .build();
```

- Uses the **builder pattern** — chain methods then call `.build()` at the end
- `.header("Content-Type", ...)` — tells FastAPI how to read the body
- `.POST(...)` — sets HTTP method to POST and attaches the body
- `.build()` — finalizes the request. nothing is sent yet

```java
HttpClient client = HttpClient.newHttpClient();
HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofStr
return response.body();
```

- `HttpClient` = the postman who delivers the request
- `client.send()` — actually sends it and waits for response
- `BodyHandlers.ofString()` — read the response as a plain String (JSON)
- `response.body()` — extract just the JSON string and return it

FILE 3: `MainController.java`

`handleAnalyze()` — Most Important Method

```java
outputArea.setText("Analyzing...");
analyzeBtn.setDisable(true);
```

- Instant feedback to user. Disables button to prevent spam clicking.

```java
new Thread(() -> {
    String json = apiService.analyzeFile(selectedFile);
```

Why a new Thread? JavaFX runs everything on one thread called the **JavaFX Application Thread**. HTTP requests take time. If we called `analyzeFile()` on the main thread, the entire UI would **freeze** until the response came back. By spawning a new thread, the HTTP work happens in the background while the UI stays responsive.

```java
javafx.application.Platform.runLater(() -> {
    outputArea.setText(formatted);
});
```

Why `Platform.runLater()`? JavaFX rule: only the JavaFX Application Thread can update the UI. But we're on a background thread. `Platform.runLater()` safely hands the UI update back to the correct thread. Without this you'd get an `IllegalStateException` crash.

`formatResponse()` — JSON to Readable Docs

```java
JsonObject root = JsonParser.parseString(json).getAsJsonObject();
```

- `JsonParser.parseString()` — converts raw JSON string into a navigable Gson object
- `.getAsJsonObject()` — so we can access fields by name like `root.get("status")`

```java
JsonArray data = root.getAsJsonArray("data");
for (JsonElement el : data) {
    JsonObject item = el.getAsJsonObject();
    String type = item.get("type").getAsString();
```

- Loops through every function/class in the data array
- Extracts type, name, line number, docstring from each item
- Formats them into Markdown-style readable text using `StringBuilder`

```java
if (i > 0) sb.append(", ");
sb.append(params.get(i).getAsString());
```

- Classic trick to avoid a leading comma — only adds `, ` before the second parameter onwards
- Result: `param1, param2, param3` not `, param1, param2`

Key Concepts Summary (for viva/presentation)

Concept	What it means in this project
AST (Abstract Syntax Tree)	A tree representation of source code structure — used to extract functions and classes without running the code
REST API	The communication style between JavaFX and FastAPI — using HTTP requests and JSON responses
Multipart Form Data	The format used to send files over HTTP
Threading in JavaFX	Background thread for HTTP calls so UI doesn't freeze
Platform.runLater()	Safe way to update JavaFX UI from a background thread
FastAPI	Python web framework that creates the backend server
Gson	Java library for parsing JSON responses from the backend
CORS	Security policy — we disabled it to allow JavaFX to talk to the backend

---

How to Run the Project

Step 1 — Start the backend:

```bash
cd docgen/backend
python -m uvicorn main:app --reload
```

Step 2 — Run the frontend:

- Open `DocGenUI` in IntelliJ
- Run `MainApp.java`

Step 3 — Use the app:

1. Click "Select Python File" and choose any `.py` file
2. Click "Analyze File"
3. Documentation appears in the text area
4. Click "Export as Markdown" to save it

Built by Zuhair — SE Semester Project, Dawood University